

Title:
Authors: Mohamed Mdhaftar
Published in: 25 may 2025

Name:
Engineering Research Paper
Question-Answer Form

- What is your take-away message from this paper?

While reading this paper, ***I discovered a compelling new application for LLMs***, it's their great power of finding complex API rules for deep-learning libraries (Tensorflow and Pytorch), although these API's are written in a combination of a low level C++ backend (like Cuda and CuDNN frameworks) and Python all refined with many optimization rules to be compatible with many hardware constraints such as homogenous and heterogenous systems (including CPU's, GPU's, TPU's, NPU's, FPGA's)-using a zero-shot learner (without fine-tuning) and a complex validation rules we can overcome the limitations of LLM's including hallucination, the repetition and the falseness and reach the limit of engineering breakthroughs.

- What is the motivation for this work (both people problem and technical problem), and its distillation into a research question? *Why doesn't the people problem have a trivial solution? What are the previous solutions and why are they inadequate?*

Right now, dealing with deep learning libraries remains a critical problem for many developers who suffer from a lack of API documentation, the unknown implementation of functions, surprising and unexplainable outputs that pop up out of nowhere, and incompatibilities regarding different hardware constraints. As a corollary, deep learning engineers struggle with weak models, unexpected logs, and sometimes false accuracy metrics.

Therefore, let's set a goal what about building an intelligent system that can automatically detect potential bugs in these libraries and then report issues to fix them?

Challenge accepted, but how can we achieve this in a highly effective and promising way?

We can't just solve this by trying to find invalid input to an API that can cause a crash or an error because even a simple function can have multiple complex relationship between it's inputs.

Furthermore, assuming that an input follows a certain rule it can be misleading because even if the input constraints seem invalid, the function still produces an output due to numerous hidden rules.

People have created methods to solve this problem such as ACETest, TitanFuzz, Pathfinder and many other tools but these tools seem to fail due to several things:

- ACETest identifies input validation path by traversing backward from error handling function calls, but error handling code can be reached after a long sequence of function calls (for loops).
- Pathfinder uses lightweight concolic testing; it inductively refines path conditions by input executions but fails due to the same reason as ACETest.
- Pathfinder computes and solves the path conditions to generate every input. Which is slow and inefficient.

- What is the proposed solution (hypothesis, idea, design)? *Why is it believed it will work? How does it represent an improvement? How is the solution achieved?*

The paper introduces "Centaur" a NeuroSymbolic Fuzzer that leverages the power of large language models to test deep learning libraries and identify potential bugs, this state-of-the-art method shows its capabilities by identifying 26 bugs that 18 were confirmed by the framework maintainers that outperforms existing solutions which is an impressive result.

The solution is composed of two phases an offline and an online phase:

The offline phase needs to be done once per API. It consists of "rule generation", "constraint learning" and "abstract input generation" algorithms.

This pipeline takes as an input the corresponding API and returns "a set of corpus of abstract inputs" meaning instead of concrete inputs directly, Centaur generates abstract inputs using Z3 and saves them in a database and it then will be reused many times to generate the input instead of generating the input every time which is time consuming.

The online phase "Fuzzing", the repetitive task is done as follows, after generating the abstract inputs Centaur randomly selects input from the corpus and constructs a concrete input using Symbolic Variables in the abstract input and it uses four tensor properties to construct a tensor : (ndim, shape, dtype, range) and finally uniformly samples content from the value range.

After these two steps we got a vast number of inputs that we can feed to the API and then inspect the logs to see if there are errors and determine if bug exists or no and the operation is done as follows:

To detect a bug:

We use Oracle:

1. crash oracle:

detecting when an input raises a signal (segmentation fault, aborted, ...)

1. differential devices (CPU and GPU):

detecting inconsistencies between outputs by running on different devices

Finally, the authors view the logs and determine if a bug exists or not

During the review process:

Classify inconsistency into 3 categories:

1. NaN when only one platform generates a “Not a Number” value
2. overflow when only one platform encounters an arithmetic overflow
3. Inconsistent when there is numerical inconsistency in the outputs (meaning the same input but different output) indicating potential bugs in implementation.

The LLM’s are used to construct the API rules set which is used to construct the abstract input database, and the solution uses zero shot learner because LLM’s already know about tensor cores and python code through Stack overflow and Github ...

- *What is the author’s evaluation of the solution? What logic, argument, evidence, artifacts (e.g., a proof-of-concept system), or experiments are presented in support of the idea?*

The author highlights Centaur’s superiority in finding deep bugs and outperforms the state-of-the-art methods. Notably, it demonstrates the capability to support up to 512 APIs for PyTorch and 522 APIs for TensorFlow, which is a highly significant result.

At first, he extracts 35 API’s from PyTorch and 32 API’s from TensorFlow and manually extracted ground truth constraints to compare them with the output of Centaur.

As expected, Centaur got impressive accuracy, it has a recall of 100% of 27 API’s and precision of 100% among 22 API’s and it covers 31 (88.6% in PyTorch) and 32 (100% in TensorFlow) of the ground truth constraints.

To strengthen their argument the author compares Centaur to different state-of-the-art baselines using different metrics:

The overall validity ratio: Centaur got higher numbers compared to all baselines in both PyTorch and TensorFlow. It achieves a validity ratio of 98.2% for PyTorch and 99.24% for TensorFlow. These results are 28.2 to 77.7 percentage points higher than competing baselines. Additionally, it takes only 8.9 milliseconds on average per input, which is very fast and efficient.

In terms of code coverage, Centaur achieves a higher capability than all baselines.

As a result, Centaur has successfully found 26 bugs among Pytorch and TensorFlow that 18 were confirmed.

All experiments were conducted on an Ubuntu 22.04 server with dual AMD EPYC 9684X CPUs and 768GB of RAM.

- What is your analysis of the identified problem, idea and evaluation? *Is this a good idea? What flaws do you perceive in the work? What are the most interesting or controversial ideas? For work that has practical implications, ask whether this will work, who would want it, what it will take to give it to them, and when might it become a reality?*

Personally, I enjoyed reading the paper and I was fascinated by the ingenuity of the idea and its implementation techniques, in fact the authors successfully tackled a major problem and achieve great results using efficient and powerful tools all combined together, the most interesting part for me was how the authors overcome the limitations of both LLM's and the Z3 solvers by using algorithmic validation rules to enhance the output. Another interesting part was how they split the framework into online and offline phases to optimize the usage of the technique rather than generating output directly like traditional programs.

However, one flaw I noticed is that the LLM's are still prone to hallucination even with strong validation mechanisms and careful prompt engineering. Prompting the model with input seeds such as the hand-written perfect examples, the 9 types of mutators and random generated invalid and valid input examples can bias the LLM reasoning toward these inputs. These inputs require not only a huge knowledge about the API and careful thinking about the documentation but also can increase the likelihood of hallucination.

Considering this work, I think that developers should use this tool to not just detect bugs but to collect a massive “*Master Directory of API Rules*” to make a database that contains all the rules for all the APIs for deep learning libraries.

And after collecting the database they need to convert grammar expressions to human friendly syntax that will be used by deep learning developers, compiler engines developers, researchers and many other engineers, it can be also extended into an IDE extension and

- What are the paper's contributions (author's and your opinion)? *Ideas, methods, software, experimental results, experimental techniques...?*

The paper introduces “Centaur” which is in my opinion, the ultimate bug hunter for deep-learning libraries, this technique presents its effectiveness and reliability by combining LLM with math formulae and SMT solvers and represents a high experimental techniques to well evaluate the model and prove its power.

- What are future directions for this research (author's and yours, perhaps driven by shortcomings or other critiques)?

From my perspective, a “Fully Automated pipeline” for detecting deep Bugs in deep learning libraries synchronized with updates to the libraries, would be the future direction of this work.

In fact, those libraries change frequently because researchers continuously develop new methods for deep learning models and they always came with new tools and optimization techniques, so using the offline-phase output would be inefficient because they quickly become outdated and do not contain new modules and classes which can contain potential bugs.

As a result, this method can be the baseline for detecting deep bugs and can be used as a test to evaluate a new feature or an update before they are published.

- What questions are you left with? *What questions would you like to raise in an open discussion of the work (review interesting and controversial points, above)? What do you find difficult to understand? List as many as you can, at least three, not including questions that can be answered quickly by searching the internet.*
- *I consider removing the check for which input the rule applied to, since LLM's can determine the appropriate input for the API rule, we simply have to refine to the LLM input prompt, I think this can improve the quality of the rules and optimize the pipeline and while using this we can get rid of a whole for loop.*
- *Another misleading idea that comes to mind is that you must manually write perfect example of the rule set, 9 mutators, and valid, invalid input for each API which can be difficult and prone to errors, so I suggest adapting another technique that automatically handles this to cope with this problem.*
- A final idea, the Grammar can be limited, and it can't generalize well to all infinite number of examples rules an API can have so must find another technique to overcome this problem.
- Question: can ‘Centaur’ generalize to other deep learning libraries such as ‘jax’ and other tools ?
- Understanding: The fuzzing algo ‘the abstract input generation’ seems challenging to understand.